🌐 Web Development Lab

Navbar Design— Practical Session Guide

15 Guided Steps • Beginner–Intermediate • Estimated Time: 50 mins

Lab Overview

In this session you will build a **modern responsive navigation bar** using **HTML and CSS (Flexbox)** — all within a single HTML file. The navbar includes a brand name, navigation links, and authentication actions (Login & Sign Up).

Each step builds progressively, allowing you to **run your code after every stage** and visually confirm your progress.

> 📌 **Note:** All code will be written inside two files: `index.html and style.css`. No external frameworks are required.

Prerequisites

- A code editor (VS Code recommended)
- A web browser (Chrome recommended)
- Basic understanding of HTML structure - Gotten by attending my HTML and CSS class
- Basic CSS knowledge (selectors, properties) - By attending my class and reading the Textbooks I shared on the google classroom.

1 Create Your Project File

Objective: Set up your working file and basic HTML structure..

⏱ **5 min**

Background

Every web page starts with a proper HTML structure. This ensures the browser correctly interprets and renders your content.

Instructions

1. Create a folder named `navbar-lab`
2. Inside it, create a file named `index.html and style.css`
3. Add the basic HTML boilerplate **(On your keyboard, enter "shift + 1 + Enter)**
4. Open the file in your browser

Code Snippet

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Navbar Lab</title>
</head>
<body>

</body>
</html>
```

💡 **Tip:** Always include the viewport meta tag for responsive design

2 Add the Navbar HTML Structure

Objective: Define the structure of the navigation bar.

⏱ **3 min**

Background

HTML provides the **structure**, while CSS handles the **layout and styling**.

Instructions

1. Inside `<body>`, add the navbar structure
2. Include brand, navigation links, and buttons

Code Snippet

```
<nav class="nav">
    <div class="brandName">Field Work</div>
```

```
    <ul>
        <li>Solutions</li>
        <li>Features</li>
        <li>Resources</li>
        <li>Pricing</li>
    </ul>

    <div class="buttonDiv">
        <a href="#">Login</a>
        <button>Sign Up</button>
    </div>
</nav>
```

💡 **Tip: Use meaningful class names — they make styling easier and code readable.**

3 Link CSS to Your HTML

Objective: Prepare your file for styling. ⏱ **2 min**

Background

In professional web development, CSS is typically written in a separate file rather than inside the HTML.

This improves:

1. Code organization
2. Reusability
3. Maintainability

The HTML file handles structure, while the CSS file handles presentation..

Instructions

1. **Inside your project folder, create a new file named styles.css**
2. **Open styles.css in your editor**
3. **Go back to your index.html file**
4. **Inside the <head> section, link the CSS file using the <link> tag**
5. **Save both files**

Code Snippet

```
<link href="style.css" rel="stylesheet">
```

💡 **Tip:** Always keep your file paths correct. If your CSS file is in the same folder as your HTML file, simply use styles.css.

4 Set the Page Background

Objective: Apply a background color to the page. ⏱ 1 min

Background

Global styling helps define the visual identity of your page.

Instructions

1. Target the `main` container
2. Apply background color

Code Snippet

```
main{
  background-color:#FAFAF8 ;
}

    color: Color(0xFFFF61DC).withOpacity(0.2))),
```

💡 **Tip:** Neutral backgrounds improve readability and UI clarity..

5 Convert Navbar into a Flex Container

Objective: Enable Flexbox layout. ⏱ 3 min

Background

Flexbox allows us to **align elements horizontally and vertically** with ease. Refer to the textbook and w3school.com/flex to read more on flex boxes if you are still confused about it.

Instructions

1. Target `.nav`
2. Add `display: flex`

Code Snippet

```
.nav{
 display:flex;
}
```

💡 **Tip:** Flexbox is ideal for one-dimensional layouts like navbars.

6 Align Items Vertically

Objective: Center all navbar items vertically. ⏱ 2 min

Background

align-items controls alignment along the cross axis (vertical).

Instructions

1. Add align-items: center

Code Snippet

```
.nav{
  display:flex;
  align-items: center;
}
```

💡 **Tip:** Without this, elements may not align neatly.

7 Space Items Across the Navbar

Objective: Distribute items across the horizontal axis. ⏱ 2 min

Background

justify-content controls spacing along the main axis (horizontal). You can see various ways of justifying contents in CSS from the class material, w3school.com

Instructions

1. **Add `justify-content: space-between`**

Code Snippet

```
..nav{
  display:flex;
  align-items: center;
  justify-content: space-between;
}
```

💡 **Tip:** This pushes elements to edges — perfect for navbars.

8 Style the Navbar Background

Objective: Make the navbar visually distinct. ⏱ **3 min**

Background

Background colors help separate UI sections.

Instructions

1. Add a background color

Code Snippet

```
Add a background color
```

💡 **Tip:** Notice that the background color f the navbar differs from that of the background of the entire page. White navbars are common in modern UI design.

9 Style the Navigation Links Container

Objective: Arrange menu items horizontally.. ⏱ **3 min**

Background

Lists (ul) are vertical by default — Flexbox makes them horizontal.

Instructions

1. Target .nav ul — Here, you are targeting all **ul tags inside the .nav class**

   Apply Flexbox

Code Snippet

```
.nav ul{
  display: flex;
  gap:20px;
  list-style-type:none;
}
```

💡 **Tip: gap is the modern way to add spacing between items either horizontally or vertically**

10 Style the Anchor Links

Objective: Remove default link styling. ⏱ **2 min**

Background

Browsers apply default styles (blue color, underline).

Instructions

1. **1.** Target .nav a that is, the anchor (a) tag inside the .nav class of your html page
2. Remove underline and change color.

Code Snippet

```
.nav a {
  color:black;
  text-decoration: none;
}
```

💡 **Tip:** Always style links to match your UI theme.

11 Style the Sign Up Button

Objective: Create a primary action button. ⏱ 3 min

Background

Buttons guide user actions — they should stand out.

Instructions

1. Target .nav button
2. Apply color, size, and border

Code Snippet

```
.nav button{
  color:white;
  border-color: #FF5733; /* Let the button have a border color */
  background-color: #FF5733;
  height: 44px;
  width:108.47px;
  border-radius: 6px; /* Give the edges a bending effect (radius).
}
```

💡 **Tip:** Use consistent colors for branding (here: orange)..

12 Style the Button Container

Objective: Align Login and Sign Up neatly. ⏱ 4 min

Background

Nested Flexbox helps organize smaller UI sections. Nested simple means, putting something inside another, in this case, having a flexbox inside the main navbar flexbox

Instructions

1. **Target .buttonDiv class**
2. **Apply Flexbox**

Code Snippet

```
.buttonDiv{
  display: flex;
  align-items: center;
  gap:20px;
  margin-right:114px;
}
```

💡 **Tip:** Flexbox can be used inside Flexbox — very common pattern

13 Style the Brand Name

Objective: Emphasize the brand identity. ⏱ 8 min

Background

Here, the “brand name” can be a logo or a text, but in our example, we are using a text. Typography plays a key role in UI design. Typography here are things like font-size, font-weight (which can be bold, bolder ext), the spacing between letters, and margin is how far it is from the defining container, refer to CSS Texts and CSS margins for more details

Instructions

1. Target .brandName class of your html
2. Apply font styling

Code Snippet

```
.brandName{
 font-size: 24px;
 font-weight: bold;
 line-height: 36px;
 letter-spacing: -0.24px;
 margin-left: 114px;
}
```

💡 **Tip:** Typography plays a key role in UI design.

14 Add Spacing and Visual Balance

Objective: Improve layout spacing. ⏱ **10 min**

Background

Spacing ensures readability and clean design.

Instructions

1. Ensure margins are correctly applied
2. Observe layout balance

💡 **Tip:** Good UI = proper spacing + alignment.

Final Structure Summary

✅ **Final Check:** Refresh your browser and ensure the navbar resembles what you see in the FIgma design and on the screenshot above. I have also provided the code for the navbar, compare your code and see the differences.